

Real-Time Visualization of Fluid Simulation *

Michel Jean Joseph Donnet

December 2025

How real-time visualization is a game changer !

The new Lattice Boltzmann method provides a relevant representation of the fluid, enabling real-time visualization of fluid simulations through GPU parallelization.

Water is something that fascinate humans for a long time. Indeed, Archimedes was already using the properties of fluids for calculations such as the famous legend in which he determined the purity of gold in a crown using the properties of a fluid. Nowadays, technology has made huge advances, allowing to better understand behavior of fluids thanks to complex simulations. These simulations are very computationally intensive and require high-end hardware. However, the development of GPUs (Graphics Processing Unit) and parallel computing on GPUs is making this type of simulation more accessible to a wider audience, particularly in the fields of visual effects, video games, etc.

In general, simulations take place in two stages: computation, then visualization. Due to the complexity of the calculations, parallelizing fluid simulation is very difficult to achieve, especially on GPUs, which are highly specialized components designed to process large amounts of data in parallel by applying the same calculations to different parts of the data, as is the case in image and video processing, for example. That's why most methods perform parallelization on CPUs (central processing units) rather than GPUs due to the complexity of exploiting parallelization on GPUs.

Some approaches that enable both visualization and computation by exploiting parallelization on GPUs have already been implemented, such as the method based on NVIDIA's GVDB Voxel technology [8], which uses the specific architecture of NVIDIA GPUs to perform computation and visualization in parallel. Other methods, designed more for visual results than for simulation realism, have been developed, particularly in game engines such as Fluid Flux for Unreal Engine [1]. The proposed method will attempt to address the challenges

*Inspired by Michel Jean Joseph Donnet's bachelor's degree project [3]

posed by GPU parallelization and fluid simulation in a novel way, using fluid modeling that facilitates GPU usage.

The new method developed and implemented by Dr. Moritz Lehmann [5], called the lattice Boltzmann method (LBM), is not based on the famous Navier-Stokes equations, which represent the flow of a viscous fluid. Indeed, these equations are very complex and have no known mathematical solution to date: the Navier-Stokes equations are among the *Millennium Prize Problems* for which a reward of one million dollars is offered to anyone who can solve them. The approach used is derived from the *Lattice Gas Automata* method developed in 1973 by J. Hardy, Y. Pomeau, and O. Pazzis [4], which uses Boltzmann equations instead of Navier-Stokes equations. In contrast to the Navier-Stokes equations, which describe the behavior of fluids at a macroscopic level, Boltzmann's equations use a microscopic representation of the fluid in the form of a set of populations of particles that interact with each other as shown on figure 1.

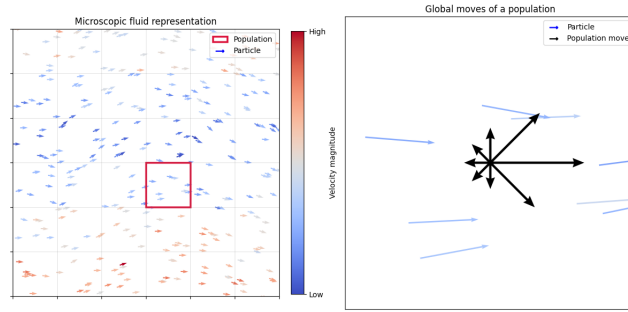


Figure 1: Lattice vectors used in the simulation

As in Boltzmann's equations, the LBM method defines a distribution function $f(\xi, \mathbf{x}, t)$ ¹ that describes the number of particles at position $\mathbf{x}$ at time t moving in direction ξ at velocity $\|\xi\|_2$ ². The main difference here is that the LBM method works with particle populations instead of considering each particle separately, which is computationally intensive. The distribution function f is normalized such that integration over velocity and position gives the mass of the population, as shown in equation (1), to ensure that the Euler's principle of conservation of mass is respected. Indeed, when the velocity and position of each particle are neglected, only the mass of the population remains.

$$\int d^3\xi \int d^3\mathbf{x} f(\xi, \mathbf{x}, t) = M(t) \quad (1)$$

¹Note that in the literature, since we are working with discrete ξ_i due to the population decomposition, $f(\xi_i, \mathbf{x}, t)$ can also be written as $f_i(\mathbf{x}, t)$.

²All variables descriptions can be found in Table 1 in Appendix

Calculating the moments of the distribution function f enables the characteristics of the fluid to be extracted, such as the particle density ρ for moment 0 or the momentum ρu for moment 1. The function f evolves over time as described in equation (2), in accordance with Newton's second law.

$$\begin{aligned}\frac{df(\xi, \mathbf{x}, t)}{dt} &= \left(\frac{dt}{dt} \cdot \frac{\partial}{\partial t} + \frac{d\mathbf{x}}{dt} \cdot \frac{\partial}{\partial \mathbf{x}} + \frac{d\xi}{dt} \cdot \frac{\partial}{\partial \xi} \right) f(\xi, \mathbf{x}, t) \\ &= \left(\frac{\partial}{\partial t} + \xi \cdot \frac{\partial}{\partial \mathbf{x}} + \frac{F}{\rho} \cdot \frac{\partial}{\partial \xi} \right) f(\xi, \mathbf{x}, t) \\ &= \frac{\partial}{\partial t} f + \xi \cdot \nabla_{\mathbf{x}} f + \frac{F}{\rho} \cdot \nabla_{\xi} f\end{aligned}\tag{2}$$

The collision operator Ω can then be defined as the relaxation of the function f towards the equilibrium function f^{eq} (defined in equation (4)) thanks to the work of Bhatnagar, Gross, and Krook[2], with τ being the relaxation time as indicated in equation (3).

$$\Omega(f) = -\frac{1}{\tau}(f - f^{eq})\tag{3}$$

$$f^{eq}(\xi, \mathbf{x}, t) = \rho \left(\frac{1}{2\pi RT} \right)^{3/2} e^{-\frac{|\xi|^2}{2RT}}\tag{4}$$

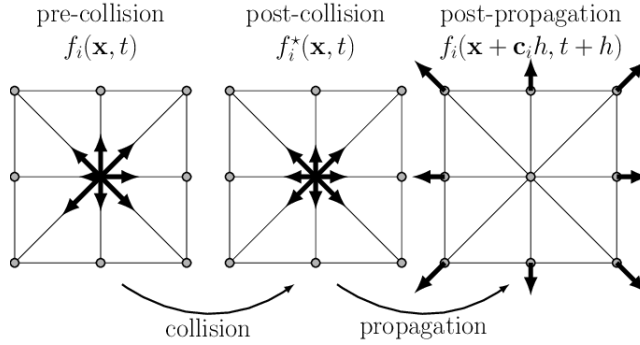
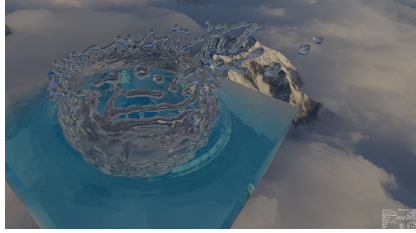


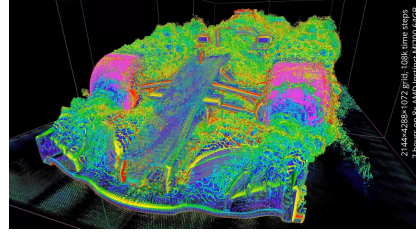
Figure 2: Basic calculation steps of the algorithm, based on the work of Schiller, Ulf and Krüger, Timm and Henrich, and Oliver [7]

As shown in Figure 2, the algorithm operates in two main steps: collision calculation, followed by propagation of the result to neighboring particle populations. In this way, interactions are local and can be exploited for GPU parallelization: the computation can be divided and distributed to each processing unit of the GPU. The implementation, benchmarks, documentation, and examples are available on ProjectPhysX/FluidX3D on GitHub [6]. GPU parallelization is effective and enables realistic real-time rendering of fluid simulations, as shown in Figure 3a. For more complex cases, the simulation can

be run on multiple GPUs from different manufacturers, as shown in Figure 3b. However, real-time visualization is not possible and the simulation must be saved.



(a) Real-time visualization of a raindrop with a resolution grid of 3 million cells



(b) Aerodynamic fluid simulation of an F1 car with a resolution of one billion cells

Figure 3: Results of Lattice Boltzmann Method

In conclusion, by approaching fluid simulation from a different angle, the Lattice Boltzmann Method allows for extensive parallelization of fluid simulation across multiple GPUs, with the possibility of real-time visualization when running on a single GPU. A next step could be to enable real-time simulation rendering on multiple GPUs, or to use a sparse volumetric representation of the data on the GPU to enable simulation with an adaptive domain and increase simulation efficiency by using less data on the GPU, or to implement the Lattice Boltzmann method in 3D software such as Blender. ³

³English correction performed by DeepL

Appendix

Table 1: Variables appearing in the governing equations

Variable	Description	Unit
t	Time	s
$\mathbf{x}$	Particle position	m
ξ	Particle velocity. Can be defined as $\xi = \frac{d\mathbf{x}}{dt}$	$m \cdot s^{-1}$
f	Particle distribution function	$m^{-3} \cdot s^3$
f^{eq}	Equilibrium distribution function	$m^{-3} \cdot s^3$
ρ	Fluid density	$kg \cdot m^{-3}$
F	External force	N
τ	Relaxation time	s
R	Specific gas constant	$J \cdot kg^{-1} \cdot K^{-1}$
T	Temperature	K
$M(t)$	Total mass of the system	kg

References

- [1] Fluid Flux Documentation – Imaginary Blend. URL: <https://imaginaryblend.com/2025/01/10/fluid-flux-documentation/>.
- [2] P.L. Bhatnagar, E.P. Gross, and M. Krook. A model for collision processes in gases. i. small amplitude processes in charged and neutral one-component systems. *Physical Review*, 94:511–525, 1954.
- [3] Michel Jean Joseph Donnet. Comparaison entre méthodes de simulation de fluides.
- [4] Judson Hardy, Yves Pomeau, and O. de Pazzis. Time evolution of a two-dimensional classical lattice system. *Physical Review Letters*, 31:276–279, 1973. URL: <https://api.semanticscholar.org/CorpusID:120869548>.
- [5] Moritz Lehmann. High Performance Free Surface LBM on GPUs. <https://epub.uni-bayreuth.de/id/eprint/5400/>, 2021.
- [6] Moritz Lehmann. FluidX3D, August 2022. URL: <https://github.com/ProjectPhysX/FluidX3D>.
- [7] Ulf Schiller, Timm Krüger, and Oliver Henrich. Mesoscopic modelling and simulation of soft matter. *Soft Matter*, 14, 11 2017. doi:10.1039/C7SM01711A.
- [8] Kui Wu, Nghia Truong, Cem Yuksel, and Rama Hoetzlein. Fast Fluid Simulations with Sparse Volumes on the GPU. *Computer Graphics Forum*, 37(2):157–167, May 2018. doi:10.1111/cgf.13350.